

Final Exam

Name: _____

Software Engineering — UATX — Spring 2026

Format	Pencil and paper. No computer, phone, notes, or coding agent.
Time	150 minutes.
Length	Seven questions.
Coverage	Lectures 5.1 through 10.1 (JS/TS and the browser, React, deployment, Docker, CI/CD, concurrency, security, agents), plus assignments 3 and 4.
Categories	Trace-through, spot the bug, specification, code review, short design.
Grading	Curved. The exam is intentionally hard; expect to leave some parts unfinished.

HOW THIS EXAM THINKS ABOUT AGENTS

The whole semester has leaned into coding agents as the way real engineers work, so an exam that asks you to write working code from scratch on paper would punish you for taking the course seriously. Instead, the exam is built around being able to work with agents: reading code carefully, finding bugs by inspection, writing the spec you'd hand to the agent, judging which of two implementations is better, and making design calls under uncertainty.

Question 1

This component renders a shopping cart. Work out what it puts on the screen for an empty cart (`items` is `[]`) and for a cart with two products. One of those cases shows something the developer did not intend.

```
function Cart({ items }) {  
  return (  
    <div>  
      <h2>Your cart</h2>  
      {items.length && (  
        <ul>  
          {items.map(i => <li key={i.id}>{i.name}</li>)}  
        </ul>  
      )}  
      {items.length > 0 && <button>Check out</button>}  
    </div>  
  );  
}
```

1. What is it?
2. Why does it happen?
3. How could you fix it?

Question 2

`products.review_count` is a denormalized counter kept in sync as reviews are added. Two shoppers review the same product at the same instant; `review_count` starts at 5. Both transactions run their `SELECT review_count` before either runs its `UPDATE`.

```
@app.post("/api/products/{product_id}/reviews")
def add_review(product_id: int, body: ReviewIn,
               user: User = Depends(current_user)):
    with engine.begin() as conn:
        conn.execute(
            text("INSERT INTO reviews (product_id, author_id, body) "
                 "VALUES (:p, :u, :b)",
                 {"p": product_id, "u": user.id, "b": body.text},
            )
        row = conn.execute(
            text("SELECT review_count FROM products WHERE id = :p"),
            {"p": product_id},
        ).first()
        conn.execute(
            text("UPDATE products SET review_count = :c WHERE id = :p"),
            {"c": row.review_count + 1, "p": product_id},
        )
    return {"ok": True}
```

1. How many review rows exist afterward, and what is `review_count`?
2. The handler already wraps all of this in a transaction (`engine.begin()`). Why didn't that prevent the problem?
3. Give a fix.

Question 3

This endpoint authenticates the caller and even checks account ownership before moving money.

```
class TransferRequest(BaseModel):
    user_id: int
    from_account: int
    to_account: int
    amount_cents: int

@app.post("/api/transfer")
def transfer(body: TransferRequest, user: User = Depends(current_user)):
    with engine.begin() as conn:
        acct = conn.execute(
            text("SELECT owner_id FROM accounts WHERE id = :id"),
            {"id": body.from_account},
        ).first()
        if acct.owner_id != body.user_id:
            raise HTTPException(403, "not your account")
        conn.execute(
            text("UPDATE accounts SET balance = balance - :a WHERE id = :id"),
            {"a": body.amount_cents, "id": body.from_account},
        )
        conn.execute(
            text("UPDATE accounts SET balance = balance + :a WHERE id = :id"),
            {"a": body.amount_cents, "id": body.to_account},
        )
    return {"ok": True}
```

1. Describe the security hole and give a concrete request that exploits it.
2. Give the fix.
3. There are also multiple correctness problems, separate from the security hole. Find at least one and say how you would address it.

Question 4

You are building the React frontend for a shared discussion board (the BBS app from the assignments). One page shows the messages for a board and lets the signed-in user post a new one. Other people post at the same time, so messages from others should appear without the user reloading the page. Assume a parent `<Board>` that renders a `<MessageList>` and a `<NewMessageForm>`. Do not write code -- design it:

1. What pieces of state does this page need, and which component owns each?
2. How do new messages -- the user's own and other people's -- get into the list, and how do you keep it reasonably fresh without a manual refresh? Name the approach and one cost of it.
3. What distinct UI states must the page handle?
4. Name one thing that is easy to get wrong here, and how you would avoid it.

Question 5

You are building the ticketing piece of an event app. The app hosts many events, each identified by its own `event_id`. What it has to do:

- Each event has its own fixed pool of free tickets (say 100). Any logged-in user can try to claim a ticket for a given event, and may hold at most one ticket per event (a user can hold tickets to several different events).
- A user should be able to tell whether they currently hold a ticket for a particular event.
- When a popular event's tickets drop, thousands of users hit the app within the same few seconds, and a client on a flaky connection may fire the same request more than once.

Design the API for this. Do not write code, and do not worry about exact status-code numbers. Cover all four:

1. What endpoint(s) does this feature need? For each, give the HTTP method, the URL, and one sentence on what it does and what it returns. (Identity comes from the auth token, not from anything the client sends.)
2. The most important correctness property is that you never hand out more than 100 tickets, even under the stampede. Explain the database-level mechanism that guarantees it, and one sentence on why doing the check in application code (read the count, then insert if there is room) does not.
3. A flaky-connection user fires their claim twice. How do you guarantee they end up with exactly one ticket, not two?
4. One ambiguity in the requirements above that you would settle before building, and the call you would make.

Question 6

Two versions of a component that fetches an item's details when the selected `id` changes. The user clicks quickly through several items. Pick the version you would ship.

```
// Version A
function Detail({ id }) {
  const [data, setData] = useState(null);
  useEffect(() => {
    fetch(`/api/items/${id}`).then(r => r.json()).then(setData);
  }, [id]);
  return <div>{data?.name}</div>;
}

// Version B
function Detail({ id }) {
  const [data, setData] = useState(null);
  useEffect(() => {
    let active = true;
    fetch(`/api/items/${id}`)
      .then(r => r.json())
      .then(d => { if (active) setData(d); });
    return () => { active = false; };
  }, [id]);
  return <div>{data?.name}</div>;
}
```

1. Justify in three to five sentences, describing the specific bug the other version has and the exact sequence of clicks that triggers it.
2. Both versions still share one shortcoming -- what is it?
3. Version B's cleanup sets a flag rather than aborting the request. Name one thing that approach does NOT do that a real cancellation would.

Question 7

Your pipeline: on every push to `main`, GitHub Actions builds a Docker image, pushes it to a registry, and the prod server pulls it and restarts. It works. Reason about three things:

1. The runner that builds the image and the prod server that runs it are different machines. What exactly travels from the runner to the server, through what, and why does the server never need your Dockerfile or your source code?
2. The image build runs the test suite (`RUN pytest`) inside the Dockerfile. Give one reason that is the wrong place for the tests in this pipeline, and where they belong.
3. A database password is needed by the CI job (to run tests against a real database) and by the running container in prod. Where should each come from? They are not the same place.